Contexto del Mini-Proyecto

Nombre: *Gestor de Tareas Académicas*
Funcionalidad meta (en este bloque):

- Registrar tarea (título, materia, tipo, urgencia, fecha)
- Listar tareas
- Eliminar tareas

Estructura propuesta (si aún no la tienes):

```
lib/
  main.dart
  models/
    task.dart
  pages/
    home_page.dart
    task_form_page.dart   // (nueva en 3.2)
  widgets/
    // (opcional para 3.3) custom_inputs.dart, task_item.dart
```

3.2 Formularios Web–móvil

3.2.1 Explicación breve (para clase)

- **Form + GlobalKey**: contenedor que permite **validar** campos en conjunto (_formKey.currentState!.validate()).
- **TextEditingController**: obtiene/limpia texto; ¡libéralo con dispose()!
- **TextFormField.validator**: retorna null si el campo es válido; un texto si es inválido.
- **Focus & UX**: TextInputAction, FocusScope.of(context).nextFocus(), cerrar teclado (unfocus()).
- **Responsive simple**: centrar y limitar ancho con ConstrainedBox + MediaQuery.

3.2.2 Práctica guiada: Pantalla de formulario de tarea

Nueva pantalla para registrar tarea con campos: **Título**, **Materia** y botón **Guardar**. (En 3.3 le añadimos más controles.)

A) Modelo (amplíalo un poco)

lib/models/task.dart

Dart
```
class Task {
final String title; // Título de la tarea
final String subject; // Materia
final String? type; // (3.3) Tipo de tarea: Tarea/Proyecto/Examen
final DateTime createdAt; // Creación
final DateTime? dueDate; // (3.3) Fecha de entrega
final bool? important; // (3.3) Marcador de importancia
final int? effort; // (3.3) Esfuerzo 1-5 (slider)

Task({
required this.title,
required this.subject,
this.type,
DateTime? createdAt,
this.dueDate,
this.important,
this.effort,
}) : createdAt = createdAt ?? DateTime.now();
}
```

*Si ya tenías el modelo básico, solo **agrega** los campos nuevos para 3.3. No rompen la app actual.*

B) Nueva pantalla: task_form_page.dart

lib/pages/task_form_page.dart

Dart
```
import 'package:flutter/material.dart';

/// Pantalla de formulario para crear una tarea.
```

```
/// (En 3.2: título y materia con validación y UX.
/// En 3.3: añadiremos más controles.)
class TaskFormPage extends StatefulWidget {
const TaskFormPage({super.key});

@override
State<TaskFormPage> createState() => _TaskFormPageState();
}

class _TaskFormPageState extends State<TaskFormPage> {
final _formKey = GlobalKey<FormState>();

// Controladores para los campos de texto
final _titleCtrl = TextEditingController();
final _subjectCtrl = TextEditingController();

@override
void dispose() {
_titleCtrl.dispose();
_subjectCtrl.dispose();
super.dispose();
}

void _submit() {
// 1) Ejecutar validaciones de todos los TextFormField
if (!(_formKey.currentState?.validate() ?? false)) return;

// 2) Si todo bien, cerrar la pantalla y devolver los datos mínimos.
Navigator.pop(context, {
'title': _titleCtrl.text.trim(),
'subject': _subjectCtrl.text.trim(),
// En 3.3 devolveremos más campos
});

// Feedback
ScaffoldMessenger.of(context).showSnackBar(
const SnackBar(content: Text('☑ Datos validados')),
);
}

@override
Widget build(BuildContext context) {
final isWide = MediaQuery.of(context).size.width >= 800;
```

```
return Scaffold(
appBar: AppBar(title: const Text('Nueva tarea')),
body: Center(
child: ConstrainedBox(
constraints: BoxConstraints(maxWidth: isWide ? 720 : 600),
child: Padding(
padding: const EdgeInsets.all(16.0),
child: Form(
key: _formKey, // clave para validar
child: Column(
mainAxisSize: MainAxisSize.min,
children: [
// Campo: Título
TextFormField(
controller: _titleCtrl,
decoration: const InputDecoration(
labelText: 'Título',
hintText: 'Ej. Investigar Redes Neuronales',
prefixIcon: Icon(Icons.task_alt),
border: OutlineInputBorder(),
),
textInputAction: TextInputAction.next, // saltar al siguiente
validator: (v) {
if (v == null || v.trim().isEmpty) {
return 'Escribe el título';
}
if (v.trim().length < 3) {
return 'Mínimo 3 caracteres';
}
return null;
},
onFieldSubmitted: (_) =>
FocusScope.of(context).nextFocus(), // UX
),
const SizedBox(height: 12),

// Campo: Materia
TextFormField(
controller: _subjectCtrl,
decoration: const InputDecoration(
labelText: 'Materia',
hintText: 'Ej. Inteligencia Artificial',
```

```
prefixIcon: Icon(Icons.book_outlined),
border: OutlineInputBorder(),
),
validator: (v) =>
(v == null || v.trim().isEmpty) ? 'Escribe la materia' : null,
onTapOutside: (_) => FocusScope.of(context).unfocus(), // cerrar teclado
),
const SizedBox(height: 16),

// Botones
Row(
children: [
Expanded(
child: OutlinedButton.icon(
onPressed: () => Navigator.pop(context),
icon: const Icon(Icons.close),
label: const Text('Cancelar'),
),
),
const SizedBox(width: 12),
Expanded(
child: FilledButton.icon(
onPressed: _submit,
icon: const Icon(Icons.save),
label: const Text('Guardar'),
),
),
],
),
],
),
),
),
),
),
);
}
}
```

C) Conectar el formulario desde HomePage

Añade un botón que abra TaskFormPage, reciba los datos y cree la tarea.

lib/pages/home_page.dart (fragmento para agregar/ajustar)

Dart

```
import 'task_form_page.dart';
import '../models/task.dart';

// ...
Future<void> _openFormAndAddTask() async {
final result = await Navigator.push(
context,
MaterialPageRoute(builder: (_) => const TaskFormPage()),
);

// result será un Map si se guardó
if (result is Map) {
final newTask = Task(
title: (result['title'] as String).trim(),
subject: (result['subject'] as String).trim(),
// 3.3 añadirá más campos aquí
);
setState(() => _tasks.insert(0, newTask));
ScaffoldMessenger.of(context)
.showSnackBar(const SnackBar(content: Text('📌 Tarea agregada')));
}
}
```

Reemplaza el onPressed del FAB por:

Dart

```
floatingActionButton: FloatingActionButton.extended(
onPressed: _openFormAndAddTask,
icon: const Icon(Icons.add),
label: const Text('Agregar'),
),
```

☑ **Checkpoint (3.2)**

- Abrir **Nueva tarea** → completar **Título** y **Materia** → **Guardar** → aparece en la **lista**.
- Validación: no permite guardar con campos vacíos.

Commit sugerido (3.2)

Shell
git checkout -b feature/formularios-3.2
git add lib/models/task.dart lib/pages/task_form_page.dart lib/pages/home_page.dart
git commit -m "3.2 Formularios: Form con validación (Título/Materia), navegación y alta de tareas"
git push -u origin feature/formularios-3.2
Mostrar más líneas

3.3 Controles (Componentes UI)

3.3.1 Explicación breve (para clase)

- **DropdownButtonFormField**: selección en lista (ej. *Tipo de tarea*).
- **RadioListTile**: selección exclusiva (ej. *Urgencia*: baja/media/alta).
- **CheckboxListTile**: *importante* sí/no.
- **SwitchListTile**: *recordatorios* on/off.
- **Slider** / **SegmentedButton**: escala de esfuerzo/estimación.
- **DatePicker**: seleccionar fecha (mostrar con showDatePicker y guardar en estado).
- **Botones y Feedback**: Elevated/Outlined/FilledButton, SnackBar, Dialog.

3.3.2 Práctica guiada: Agregar controles al formulario

Edita lib/pages/task_form_page.dart para **añadir** nuevos campos controlados por estado.

Dart
// Dentro de _TaskFormPageState:
String? _type; // Dropdown: Tarea/Proyecto/Examen
DateTime? _dueDate; // DatePicker
bool _important = false; // Checkbox

```
bool _reminder = false; // Switch
int _effort = 3; // Slider 1..5
String _urgency = 'Media'; // Radio: Baja/Media/Alta

// Helper para formatear fecha simple
String _formatDate(DateTime? d) =>
(d == null) ? 'Sin fecha' : '${d.year}-${d.month.toString().padLeft(2,'0')}-
${d.day.toString().padLeft(2,'0')}';
```

Ahora agrega los **widgets** debajo de los TextFormField:

Dart

```
// --- Tipo de tarea (Dropdown) ---
DropdownButtonFormField<String>(
value: _type,
decoration: const InputDecoration(
labelText: 'Tipo de tarea',
border: OutlineInputBorder(),
prefixIcon: Icon(Icons.category_outlined),
),
items: const [
DropdownMenuItem(value: 'Tarea', child: Text('Tarea')),
DropdownMenuItem(value: 'Proyecto', child: Text('Proyecto')),
DropdownMenuItem(value: 'Examen', child: Text('Examen')),
],
onChanged: (v) => setState(() => _type = v),
),

const SizedBox(height: 12),

// --- Urgencia (RadioListTile en columna) ---
Container(
decoration: BoxDecoration(
border: Border.all(color: Colors.grey.shade300),
borderRadius: BorderRadius.circular(8),
),
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
const ListTile(
title: Text('Urgencia', style: TextStyle(fontWeight: FontWeight.bold)),
),
RadioListTile<String>(
```

```
title: const Text('Baja'),
value: 'Baja',
groupValue: _urgency,
onChanged: (v) => setState(() => _urgency = v!),
),
RadioListTile<String>(
title: const Text('Media'),
value: 'Media',
groupValue: _urgency,
onChanged: (v) => setState(() => _urgency = v!),
),
RadioListTile<String>(
title: const Text('Alta'),
value: 'Alta',
groupValue: _urgency,
onChanged: (v) => setState(() => _urgency = v!),
),
],
),
),

const SizedBox(height: 12),

// --- Importante (Checkbox) & Recordatorio (Switch) ---
CheckboxListTile(
value: _important,
onChanged: (v) => setState(() => _important = v ?? false),
title: const Text('Marcar como importante'),
secondary: const Icon(Icons.priority_high_outlined),
controlAffinity: ListTileControlAffinity.leading,
),

SwitchListTile(
value: _reminder,
onChanged: (v) => setState(() => _reminder = v),
title: const Text('Activar recordatorio'),
secondary: const Icon(Icons.notifications_active_outlined),
),

const SizedBox(height: 12),

// --- Esfuerzo (Slider 1..5) ---
Column(
```

```
crossAxisAlignment: CrossAxisAlignment.start,
children: [
const Text('Esfuerzo estimado'),
Row(
children: [
const Icon(Icons.timer_outlined),
Expanded(
child: Slider(
value: _effort.toDouble(),
min: 1,
max: 5,
divisions: 4,
label: '$_effort',
onChanged: (v) => setState(() => _effort = v.toInt()),
),
),
],
),
],
),

const SizedBox(height: 12),

// --- Fecha de entrega (DatePicker) ---
Row(
children: [
Expanded(
child: OutlinedButton.icon(
icon: const Icon(Icons.calendar_month),
label: Text('Fecha entrega: ${_formatDate(_dueDate)}'),
onPressed: () async {
final now = DateTime.now();
final picked = await showDatePicker(
context: context,
firstDate: DateTime(now.year - 1),
lastDate: DateTime(now.year + 2),
initialDate: _dueDate ?? now,
);
if (picked != null) setState(() => _dueDate = picked);
},
),
),
const SizedBox(width: 12),
```

```
IconButton(
tooltip: 'Borrar fecha',
onPressed: () => setState(() => _dueDate = null),
icon: const Icon(Icons.clear),
),
],
),
```

Y **envía** estos valores al guardar (en _submit()):

Dart
```
Navigator.pop(context, {
'title': _titleCtrl.text.trim(),
'subject': _subjectCtrl.text.trim(),
'type': _type,
'urgency': _urgency,
'important': _important,
'reminder': _reminder,
'effort': _effort,
'dueDate': _dueDate?.toIso8601String(),
});
```

Actualiza el receptor en **HomePage**:

Dart
```
if (result is Map) {
final newTask = Task(
title: (result['title'] as String).trim(),
subject: (result['subject'] as String).trim(),
type: result['type'] as String?,
dueDate: (result['dueDate'] != null)
? DateTime.parse(result['dueDate'] as String)
: null,
important: result['important'] as bool?,
effort: result['effort'] as int?,
// urgency es solo de UI; podrías guardarla si la añades al modelo
);
setState(() => _tasks.insert(0, newTask));
}
```

Nota: Si deseas **persistir urgency** en el modelo, agrega final String? urgency; y ajústalo igual que los demás campos.

☑ Checkpoint (3.3)

- El formulario ahora permite definir **Tipo**, **Urgencia**, **Importante**, **Recordatorio**, **Esfuerzo (1–5)** y **Fecha de entrega**.
- Al guardar, la tarea se crea con estos datos (los que estén en el modelo).
- La **lista** sigue mostrando al menos **título** y **materia** (puedes mejorar el subtitle para mostrar type o dueDate).

Commit sugerido (3.3)

Shell
git checkout -b feature/controles-3.3
git add lib/models/task.dart lib/pages/task_form_page.dart lib/pages/home_page.dart
git commit -m "3.3 Controles: Dropdown, Radio, Checkbox, Switch, Slider y DatePicker integrados al formulario"
git push -u origin feature/controles-3.3
Mostrar más líneas

Práctica guiada para el estudiante (entregable)

Parte A — Formularios (3.2)

1. Crear TaskFormPage con **dos campos**: Título, Materia.
2. Implementar **validación** (no vacíos, longitud mínima).
3. Conectar a HomePage para **agregar** la tarea a la lista.
4. **Evidencias**:
    - Captura del formulario **con errores de validación**.
    - Captura de la **lista** después de guardar.
5. **Commit**: 3.2 Formularios listos (validación + navegación)

Parte B — Controles (3.3)

1. Añadir al formulario:
    - Dropdown (**Tipo**: Tarea/Proyecto/Examen)
    - Radio (**Urgencia**: Baja/Media/Alta)
    - Checkbox (**Importante**)
    - Switch (**Recordatorio**)
    - Slider (**Esfuerzo 1–5**)
    - **DatePicker** (fecha de entrega)
2. Reflejar los valores en la **creación de la tarea** (al menos type, dueDate, important, effort).
3. **Evidencias**:
    - Captura del formulario **con todos los controles**.
    - Captura de la **lista** mostrando al menos Título/Materia/Tipo o Fecha (a tu elección en el subtitle).
4. **Commit**: 3.3 Controles integrados al formulario

Preguntas de comprensión (para su reporte)

1. ¿Cuál es la diferencia entre TextField y TextFormField?
2. ¿Para qué sirve GlobalKey<FormState> y cuándo se usa validate()?
3. ¿Qué patrón de estado estás usando en esta pantalla? ¿Qué limitaciones tiene?
4. ¿Cómo harías para **deshabilitar** el botón Guardar si el formulario es inválido?

Retos opcionales (incrementales)

- **Reto 1 (UX):** Mostrar dueDate como **chip** en la tarjeta de la tarea.
- **Reto 2 (Filtro):** Añadir un PopupMenuButton en AppBar para filtrar por type.
- **Reto 3 (Persistencia):** Guardar tareas en shared_preferences o hive (sincronizar con web y móvil).
- **Reto 4 (Validación avanzada):** Validar que dueDate sea **posterior** a hoy.

Sugerencia de rúbrica rápida (para estas prácticas)

Criterio	Excelente (100)	Bueno (85)	Básico (70)	Insuficiente (≤60)
3.2 Formulario + Validación	Form completo, validación robusta, UX cuidada (focus, teclado), navegación correcta	Valida y navega, detalles menores de UX	Valida lo esencial, navegación básica	Sin validación o no agrega
3.3 Controles	Todos los controles operativos y reflejados en el modelo	1–2 controles no persisten o faltan detalles	Implementa controles pero no persisten	Controles no funcionales
Evidencias & Commits	Capturas claras + commits con mensajes significativos	Capturas y commits presentes	Faltan evidencias o mensajes pobres	